

KIM-1 Assembly Language Workbook

44 problems that start with a single `LDA` and end with interrupt-driven clocks, music and graphics on the 6502, the 6530 RRIOTs and the MTU K-1008 Visible Memory.



Every answer in this book assembles in Kimulator's built-in assembler and is checked by a test that runs it on the emulated KIM-1. The source files are in `docs/workbook/answers`.

Contents

Before you start

Part 1 · First steps

- 1 Hello, memory
- 2 Copy cat
- 3 Swap
- 4 Neighbours
- 5 Change it in place
- 6 Add two bytes
- 7 Subtract two bytes
- 8 16-bit addition
- 9 16-bit subtraction
- 10 Times ten
- 11 Increment a 24-bit number
- 12 Negate a 16-bit number

Part 2 · Decisions, loops and data

- 13 The bigger of two
- 14 Counting tables
- 15 Sum a table
- 16 Smallest and largest
- 17 Masks
- 18 Count the ones
- 19 Multiply
- 20 Byte to hex text
- 21 Block copy
- 22 SHOUT
- 23 Decimal addition

- 24 Binary to decimal

- 25 Sort

Part 3 · The KIM-1: display, keypad, timers and sound

- 26 Say something
- 27 Calculator entry
- 28 Tally counter
- 29 HELLO
- 30 Marquee
- 31 Read the keypad yourself
- 32 Egg timer
- 33 Interrupt-driven stopwatch
- 34 Concert A
- 35 Play a tune
- 36 Keypad organ

Part 4 · The K-1008 Visible Memory

- 37 Paint the screen
- 38 Checkerboard
- 39 Plot a dot
- 40 Draw a line
- 41 Hello, world
- 42 Scroll up
- 43 Bouncing ball
- 44 Sketch pad

Answers

Appendix · 6502 instruction summary

Before you start

This workbook teaches 6502 assembly language on the KIM-1. Each part adds a few instructions or a piece of hardware, then gives you problems that use them. Early problems take three or four instructions. The last ones take a page. Try each problem before you turn to its answer at the back. There is usually more than one good solution, so an answer that differs from the book's can still be right.

Working in Kimulator

- **View › Assembler** (Ctrl+E) opens the editor. **F7** assembles, **F8** loads the program into memory and makes it start the monitor's open cell, and **F5** loads and runs it. The program starts at the `start` label.
- **View › Debugger** (Ctrl+D) shows the registers and a memory editor. Use it to put test values into memory before a run and to check the results afterwards. You can also use the keypad: **AD**, an address, **DA**, a value, then **+** for the next cell.
- Programs in Parts 1 and 2 end with `BRK`. With **Machine › Preset NMI/IRQ vectors at power-on** turned on (the default), `BRK` returns to the monitor. On a real KIM-1, store `$00` at `$17FE` and `$1C` at `$17FF` first, so that `BRK` goes to the monitor.
- Programs in Parts 3 and 4 run until you stop them with **ST** (Esc) or **RS** (F12).

Conventions

- Programs start at `$0200`. Inputs and outputs are in zero page from `$10` up, unless a problem says otherwise.
- The KIM-1 has 1K of RAM, `$0000` – `$03FF`. Page 1 (`$0100` – `$01FF`) is the stack and the monitor uses `$EF` – `$FF`. That leaves zero page `$00` – `$EE` and `$0200` – `$03FF` for you, plus 64 bytes at `$1780` – `$17BF`.
- “Byte” means 8 bits. 16-bit numbers are stored *little-endian*: low byte first. “(`$11:$10`)” means the 16-bit number with its high byte at `$11` and its low byte at `$10`.

Assembler syntax

Kimulator's assembler uses modern ca65/64tass-style syntax. It knows all the KIM-1 monitor's names (`SCANDS`, `POINTL`, `SAD`, ...), so you don't have to define them.

```
.org $0200           ; where the following code goes
COUNT              = 10           ; a constant
start:  ldx #COUNT      ; label, instruction, operand; # = immediate (the number itself)
@loop:  dex              ; @local labels belong to the previous normal label
        bne @loop
        lda #<MSG        ; < low byte, > high byte of a 16-bit value
        lda #'A'         ; a character's ASCII code
        and #%00001111    ; $hex, %binary, plain decimal
        brk
MSG:     .byte "HI", $0D, 0 ; data: .byte, .word (16-bit, little-endian)
LEN      = * - MSG        ; * is the current address
```

Part 1 • First steps

The 6502 has three 8-bit registers you work with directly. **A**, the accumulator, does the arithmetic. **X** and **Y** are index registers, used for counting and stepping through tables. The **program counter** (PC) holds the address of the next instruction. The **status register** (P) holds flags that instructions set as a side effect. The ones you need first:

Flag	Set when...
C carry	an addition carries out of bit 7, or a subtraction does <i>not</i> need to borrow
Z zero	the result is zero
N negative	bit 7 of the result is 1

An instruction's **addressing mode** tells the CPU where its operand is:

Mode	Example	Operand	Bytes
Immediate	LDA #\$42	the number \$42 itself	2
Zero page	LDA \$10	the byte at \$0010	2
Absolute	LDA \$0300	the byte at \$0300	3
Implied	INX	(built into the instruction)	1

The most common beginner's mistake is to forget the **#**. `LDA #$10` loads the number 16, but `LDA $10` loads whatever is *stored at* address \$0010.

The instructions in Part 1:

Instructions	What they do
LDA LDX LDY	load a register from memory (or an immediate value); set Z and N
STA STX STY	store a register into memory; flags unchanged
TAX TAY TXA TYA	copy between A and X or Y
INX INY DEX DEY	add or subtract 1 in X or Y (\$FF + 1 wraps to \$00)
INC DEC	add or subtract 1 in memory
CLC SEC	clear or set the carry
ADC	A = A + operand + C. Clear the carry first for a plain addition
SBC	A = A - operand - (1 - C). Set the carry first for a plain subtraction
BRK	stop and return to the monitor

Carry as a chain. The 6502 only adds 8 bits at a time. To add bigger numbers, add the low bytes first, then the next bytes *with* the carry left by the step before, and so on. That is why there is no “add without carry” instruction: CLC at the start of the chain gives you one.

1 Hello, memory

New: `LDA` `STA` `BRK`

Store the value `$42` in memory location `$10`.

Try it: after the run, `$10` holds `$42`.

2 Copy cat

Copy the byte at \$10 into \$11, \$12 and \$13. How few instructions can you do it in?

Try it: \$10 = \$9C → \$11 = \$9C, \$12 = \$9C, \$13 = \$9C.

3 Swap

New: LDX LDY STX STY

Exchange the bytes at \$10 and \$11. Don't use the accumulator.

Try it: \$10 = \$12, \$11 = \$AB → \$10 = \$AB, \$11 = \$12.

4 Neighbours

New: TAX TAY TXA TYA INX INY DEX DEY

A number N is at \$10. Store N + 1 at \$11 and N - 1 at \$12. Read N from memory only once, into A, and copy it from there.

Try it: N = \$41 → \$42, \$40. What do you get for N = \$FF and N = \$00?

5 Change it in place

New: INC DEC

Add 2 to the byte at \$10 and subtract 3 from the byte at \$11, without using A, X or Y.

Try it: \$10 = \$FF, \$11 = \$02 → \$01, \$FF.

6 Add two bytes

New: CLC ADC

Store \$10 + \$11 at \$12. Ignore a carry out of the result. Why does the program need CLC even if the carry is usually clear already?

Try it: \$12 + \$34 = \$46; \$F0 + \$20 = \$10 (the carry is lost).

7 Subtract two bytes

New: SEC SBC

Store \$10 - \$11 at \$12.

Try it: \$50 - \$20 = \$30; \$10 - \$20 = \$F0, which is -16 in two's complement.

8 16-bit addition

Add the 16-bit numbers (\$11:\$10) and (\$13:\$12) and store the sum in (\$15:\$14).

Try it: \$12FF + \$0001 = \$1300: put FF 12 01 00 at \$10-\$13 and expect 00 13 at \$14-\$15.

9 16-bit subtraction

Store $(\$11:\$10) - (\$13:\$12)$ in $(\$15:\$14)$.

Try it: $\$1300 - \$0001 = \$12FF$; $\$0000 - \$0001 = \$FFFF$.

10 Times ten

The 6502 has no multiply instruction. N (0–25) is at $\$10$. Store $10 \times N$ at $\$11$ using only additions. Aim for fewer than ten additions.

Hint: $2N$ is $N + N$. Keep a partial result in a spare memory location.

Try it: $N = 25 \rightarrow 250$ ($\$FA$).

11 Increment a 24-bit number

Add 1 to the 24-bit number at $\$10$ (low), $\$11$, $\$12$ (high). You haven't met any branch instructions yet, so the program has to run straight through.

Hint: `ADC #0` adds nothing but the carry.

Try it: $FF\ FF\ 12 \rightarrow 00\ 00\ 13$; $FF\ FF\ FF \rightarrow 00\ 00\ 00$.

12 Negate a 16-bit number

Store $-(\$11:\$10)$, in two's complement, in $(\$13:\$12)$.

Hint: $-x = 0 - x$.

Try it: $1 \rightarrow \$FFFF$; 1000 ($\$03E8$) $\rightarrow \$FC18$; $\$8000 \rightarrow \8000 (why?).

Part 2 · Decisions, loops and data

Comparing and branching

`CMP`, `CPX` and `CPY` subtract memory from a register without storing the result. They only set the flags. A **branch** then jumps if a flag is in a given state. Branches reach about 127 bytes forwards or backwards. For anything farther, use `JMP`.

After <code>CMP value</code> , if A is...	then	so branch with	Branch	if
equal to value	Z = 1	<code>BEQ</code> (BNE if not)	<code>BEQ BNE</code>	Z = 1 / 0
greater than or equal (unsigned)	C = 1	<code>BCS</code>	<code>BCS BCC</code>	C = 1 / 0
less than (unsigned)	C = 0	<code>BCC</code>	<code>BMI BPL</code>	N = 1 / 0
			<code>BVS BVC</code>	V = 1 / 0

Loads, transfers, `INX / DEX`, logic and arithmetic all set Z and N, so a loop that counts down to zero doesn't need a compare: `DEX` followed by `BNE` repeats until X reaches 0.

Indexed and indirect addressing

Mode	Example	Address used
Absolute,X / ,Y	<code>LDA \$0300,X</code>	<code>\$0300 + X</code>
Zero page,X	<code>LDA \$10,X</code>	<code>\$10 + X</code> (stays in zero page)
Indirect indexed	<code>LDA (\$10),Y</code>	(the 16-bit address stored at <code>\$10/\$11</code>) + Y
Indexed indirect	<code>LDA (\$10,X)</code>	the 16-bit address stored at <code>\$10 + X</code>
Indirect	<code>JMP (\$17FE)</code>	<code>JMP</code> only: to the address stored at <code>\$17FE/\$17FF</code>

`(zp),Y` is the 6502's pointer. Keep a 16-bit address in two zero-page bytes, and you can reach any byte in memory. X and Y only go up to 255, so for bigger blocks, step the high byte of the pointer one page at a time.

Logic and shifts

Instruction	Effect	Typical use
<code>AND</code>	A = A and operand	clear bits (keep only the bits that are 1 in the mask)
<code>ORA</code>	A = A or operand	set bits
<code>EOR</code>	A = A exclusive-or operand	flip bits
<code>BIT</code>	Z = (A and operand) = 0; N = bit 7 and V = bit 6 of the operand	test bits without changing A
<code>ASL / LSR</code>	shift left / right; the bit shifted out goes to C, a 0 comes in	$\times 2$, $\div 2$
<code>ROL / ROR</code>	rotate left / right through C: the old C comes in	multi-byte shifts

Shifts and rotates work on A (`ASL A`) or directly on memory (`ASL $10`).

The stack and subroutines

`JSR label` pushes the return address on the stack (page 1) and jumps, and `RTS` returns. `PHA / PLA` push and pull A, and `PHP / PLP` do the same for the flags. Every push needs a matching pull before the `RTS`. Otherwise the subroutine returns to the wrong place.

Decimal mode

After `SED`, `ADC` and `SBC` treat each byte as two decimal digits (BCD): $\$19 + \$01 = \$20$, and the carry means “over 99”. `CLD` switches back. The KIM-1 display shows hex digits, so a BCD number shows up in decimal on the display.

13 The bigger of two

New: `CMP` `BCS` `BCC`

Store the larger of the unsigned bytes at `$10` and `$11` in `$12`.

Try it: 3, 9 → 9; 200, 9 → 200; 7, 7 → 7.

14 Counting tables

New: `CPX` `BNE` `BPL` and absolute,X / absolute,Y addressing

Fill `$0300 – $033F` with `$00`, `$01`, ..., `$3F` and `$0340 – $037F` with `$3F`, `$3E`, ..., `$00`.

Hint: For the second table, count one register down while the other counts up.

15 Sum a table

Add up the 32 bytes at `$0300 – $031F`. Store the 16-bit sum at `$10` (low) and `$11` (high).

Try it: fill the table with `$FF` → `$1FE0` (`E0 1F`).

16 Smallest and largest

`$0300` holds a count N (1–255), followed by N unsigned bytes. Store the smallest at `$10` and the largest at `$11`.

Try it: 05 09 08 07 0A 03 at `$0300` → 3 and 10.

17 Masks

New: `AND` `ORA` `EOR` `BIT` `BVC` `BVS`

From the byte at `$10`, work out:

- `$11` = its low four bits (low nibble),
- `$12` = the byte with bits 7 and 0 set,
- `$13` = the byte with every bit inverted,
- `$14` = 1 if bit 6 is set, otherwise 0.

Try it: `$5A` → `$0A`, `$DB`, `$A5`, `$01`.

18 Count the ones

New: LSR

Store the number of 1 bits in the byte at \$10 in \$11, leaving \$10 unchanged. Can your loop stop as soon as there are no 1 bits left?

Try it: \$5A → 4; \$FF → 8; \$00 → 0.

19 Multiply

New: ASL ROL

Multiply the unsigned bytes at \$10 and \$11 and store the 16-bit product in (\$13:\$12).

Hint: Do it the way you multiply on paper, but in binary. For each 1 bit in the multiplier, add the multiplicand, shifted left by that bit's position. A 16-bit value doubles with ASL on the low byte followed by ROL on the high byte.

Try it: $200 \times 3 = 600$ (\$0258); $255 \times 255 = \text{\$FE01}$.

20 Byte to hex text

New: JSR RTS PHA PLA

Convert the byte at \$10 to two ASCII characters: the high hex digit at \$11, the low one at \$12. Write a subroutine NIBBLE that turns a value 0–15 in A into its ASCII digit, and call it twice.

Hint: ASCII '0' – '9' are \$30 – \$39; 'A' – 'F' are \$41 – \$46.

Try it: \$3C → \$33, \$43 ("3C").

21 Block copy

New: CPY and indirect indexed addressing, (zp),Y

Copy a block of bytes. The source address is at \$10 / \$11, the destination address at \$12 / \$13 and the count (1–255) at \$14. The two blocks don't overlap.

Try it: copy 16 bytes from \$0300 to \$0380: 00 03 80 03 10 at \$10.

Going further: *What goes wrong if the blocks overlap and the destination is above the source? How would you fix it?*

22 SHOUT

\$10 / \$11 points to a string of ASCII characters ended by a zero byte. Change its lowercase letters to uppercase in place, and store the string's length in \$12.

Hint: Look at 'a' (\$61) and 'A' (\$41) in binary.

Try it: "Hello, KIM-1!" → "HELLO, KIM-1!", length 13. Put the string in with .byte "Hello, KIM-1!", 0.

23 Decimal addition

New: SED CLD

\$10 / \$11 and \$12 / \$13 each hold a 4-digit BCD number, low byte first, so 1234 is stored as 34 12. Store their sum in \$14 / \$15 and the carry (the ten-thousands digit) in \$16.

Try it: $5678 + 6789 = 12467$: 67 24 and 01.

24 Binary to decimal

Convert the byte at \$10 (0–255) to decimal: the hundreds digit at \$11 and the tens and units as packed BCD at \$12. You will need this to show numbers on the display in decimal.

Hint: Take the bits from the top down. For each one, $\text{result} = 2 \times \text{result} + \text{bit}$. If you do the doubling with ADC in decimal mode, the result is already BCD.

Try it: 255 → 02, 55; 99 → 00, 99; 100 → 01, 00.

25 Sort

\$0300 holds a count N (2–255), followed by N unsigned bytes. Sort them into ascending order, in place.

Hint: Bubble sort. Compare each neighbouring pair and swap them if they're in the wrong order. Repeat until a whole pass makes no swaps.

Try it: 05 05 03 09 01 03 → 05 01 03 03 05 09.

Going further: *After each pass, the largest remaining value has reached its final place. Use that to make every pass shorter than the one before.*

Part 3 · The KIM-1: display, keypad, timers and sound

Now for the KIM-1 itself. Its two 6530 RRIOT chips each hold 1K of ROM, 64 bytes of RAM, two 8-bit I/O ports and an interval timer. The 6530-002 runs the keypad, the display, the teletype and the tape. The 6530-003's ports go to the application connector and are yours to use.

Address	What is there
\$0000-\$03FF	1K RAM
\$1700-\$173F	6530-003 ports and timer (application)
\$1740-\$177F	6530-002 ports and timer (keypad, display)
\$1780-\$17BF	6530-003 RAM, free
\$17C0-\$17FF	6530-002 RAM; the monitor uses \$17E7 up, with the vectors NMIV \$17FA, IRQV \$17FE
\$1800-\$1BFF	6530-003 ROM: tape routines
\$1C00-\$1FFF	6530-002 ROM: the monitor

Monitor routines

Routine	Address	What it does
SCANDS	\$1F1F	Lights the six digits once, showing POINTH (\$FB), POINTL (\$FA) and INH (\$F9) as hex. That takes about 4 ms, so call it over and over. It returns with Z = 1 if no key is down and Z = 0 if one is. It changes A, X and Y.
GETKEY	\$1F6A	Returns the code of the key that is down in A: \$00-\$0F for 0-F, \$10 AD, \$11 DA, \$12 +, \$13 GO, \$14 PC, or \$15 for none. Port A must already be an input (PADD = 0), which is how SCANDS leaves it. Takes up to about 0.3 ms.
AK	\$1EFE	Is any key down? A ≠ 0 (and Z = 0) if so.

ST and RS are not keys the program can read. ST causes an NMI and RS a reset. Monitor routines expect binary mode, so always CLD before calling them.

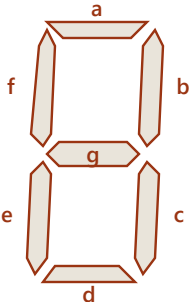
The 6530 registers

Both chips have the same registers, at \$1700 (6530-003) and \$1740 (6530-002):

Offset	Write	Read	6530-003	6530-002 (monitor names)
+0	port A data		\$1700	\$1740 SAD
+1	port A direction: 1 = output, 0 = input		\$1701	\$1741 PADD
+2	port B data		\$1702	\$1742 SBD
+3	port B direction		\$1703	\$1743 PBDD
+4	start timer, ÷1		\$1704	\$1744 CLK1T
+5	start timer, ÷8		\$1705	\$1745 CLK8T
+6	start timer, ÷64	timer value	\$1706	\$1746 CLK64T
+7	start timer, ÷1024	bit 7 = timed out	\$1707	\$1747 CLKKT / CLKRDI
+\$C ... +\$F	as +4 ... +7, with the timer interrupt enabled		\$170C-\$170F	—

The interval timer. Write a count N to one of the “start timer” addresses. The timer counts down once every 1, 8, 64 or 1024 clock cycles (μ s on a 1 MHz KIM-1). Bit 7 of offset +7 goes to 1 when it has counted N + 1 steps, so for exactly T steps, write T – 1. Writing a new count starts the timer again and clears the flag. Reading offset +7 doesn't clear it. After the time-out, the counter keeps going down at the full clock rate, so the program has to restart the timer itself.

The display



Segments a–g are PA0–PA6.

Each digit's segments a–g are wired to PA0–PA6 of the 6530-002 (SAD). A 1 lights a segment once `PADD = $7F`. PB1–PB4 (SBD, with `PBDD = $1E`) feed a 74145 decoder whose outputs 4–9 select digits 1–6, so write $(n + 4) \times 2$ to SBD to select digit n, counting 0–5 from the left. Only one digit is lit at a time. Show each one for about half a millisecond and go round again, and your eye sees them all at once.

Shows	Segments	Byte	Shows	Segments	Byte
H	b c e f g	\$76	L	d e f	\$38
E	a d e f g	\$79	0 / 0	a b c d e f	\$3F

The keypad

C	D	E	F
8	9	A	B
4	5	6	7
0	1	2	3

The 23 keys form a matrix. Write the row number $\times 2$ to SBD, and the 74145 pulls that row low (outputs 0–2). With `PADD = 0`, a key that is down in that row pulls its column's port A bit to 0:

Row	PA6	PA5	PA4	PA3	PA2	PA1	PA0
0	0	1	2	3	4	5	6
1	7	8	9	A	B	C	D
2	E	F	AD	DA	+	GO	PC

So a key's code is $\text{row} \times 7 + (6 - \text{bit number})$. PA7 is the teletype input, not a key.

Interrupts

An IRQ makes the 6502 finish its current instruction, push PC and P on the stack, set the I flag (so it isn't interrupted again straight away) and jump through the vector at `$FFFE`. On the KIM-1 that vector is in the monitor ROM, which jumps on through `IRQV` at `$17FE / $17FF`, so the handler's address goes there. The handler saves any register it uses, deals with the cause (for the timer, it writes a new count, which also clears the interrupt), restores the registers and ends with `RTI`. `RTI` restores P and PC. `SEI` and `CLI` turn interrupts off and on. `BRK` uses the same vector.

The 6530-003 timer's interrupt comes out on its PB7 pin. On a real KIM-1 you wire PB7 to the IRQ line yourself. In Kimulator, turn on **Machine > Timer IRQ jumper**.

Sound

The KIM-1 has no speaker. People connected an amplifier to a port pin and switched it on and off fast enough to be heard. A square wave of frequency f flips the pin every $1/(2f)$ seconds. In Kimulator, pick **Machine › Sound › Application port PA0** to hear PA0 of the 6530-003.

Note	C4	D4	E4	F4	G4	A4	B4	C5
Hz	261.63	293.66	329.63	349.23	392.00	440.00	493.88	523.25
Note	D5	E5	F5	G5	A5	B5	C6	D6
Hz	587.33	659.26	698.46	783.99	880.00	987.77	1046.50	1174.66

26 Say something

New: JSR SCANDS , POINTH POINTL INH , JMP

Make the display show **C0DE 42** until you press ST or RS.

27 Calculator entry

New: JSR GETKEY

Start with **0000 00**. Each hex key (0–F) shifts the six digits one place left and appears at the right, like a pocket calculator. Ignore the other keys. One press must give exactly one digit, however long the key is held down.

Hint: After handling a key, keep calling SCANDS until it says no key is down.

Try it: type 1 2 3 4 5 6 → **1234 56**.

28 Tally counter

Count key presses in decimal, 0000–9999, in the four address digits, and show the code of the last key pressed in the data digits.

Try it: press 3 eleven times, then + → **0012 12**.

29 HELLO

Show **HELLO** on the first five digits (the sixth blank) without any monitor routines. Drive the ports yourself.

Hint: Blank the segments before you switch to the next digit. Otherwise you see a faint ghost of each letter on its neighbour.

30 Marquee

Scroll the message **HELLO 6502**, followed by two blanks, across the display from right to left, one step every quarter of a second, forever. Use the 6530-003 timer, polled between passes over the digits, to time the steps.

Hint: A quarter second is 250,000 μ s, or about 244 steps of the $\div 1024$ timer.

31 Read the keypad yourself

Write `READKEY`, your own version of `GETKEY`. It returns the same codes, and `$15` when no key is down, by scanning the rows through the ports. Test it with a main loop that shows the last key's code in the data digits (use `SCANDS` for the display).

Try it: press DA → `0000 11`; press 7 → `0000 07`.

32 Egg timer

Count down from a time in minutes and seconds, kept as two BCD bytes `MINS` and `SECS` in the program. Show `MMSS 00`, and at zero stop and show `0000 EE`. Keep time in quarter seconds by polling the 6530-003 timer between calls to `SCANDS`.

Hint: When the seconds go below zero in BCD, they should become 59 and the minutes should go down by one.

Going further: *Restart the timer at the end of the loop instead of as soon as the flag is set, and time a minute. Why is it slower?*

33 Interrupt-driven stopwatch

New: `SEI` `CLI` `RTI`, `IRQV`

Show `MMSS hh`: minutes, seconds and hundredths. Any key starts and stops the watch. A timer interrupt every 10 ms does the counting, so the display loop only shows the time and reads keys.

Setup: Machine > Timer IRQ jumper on.

Hint: 10 ms is 156.25 steps of the ÷64 timer. What happens to the D flag when the interrupt arrives while the main program is in binary mode?

34 Concert A

Play a steady 440 Hz tone on PA0 of the application port.

Setup: Machine > Sound > Application port PA0.

Hint: Start the timer for the next half period before you flip the pin. Then the time your loop takes hardly matters.

35 Play a tune

Play a tune from a table of (pitch, length) byte pairs. The pitch is the half period in ÷8 timer steps, with 0 for a rest. The length is in 1/16 s units, and 0 ends the tune. Time the pitch with the 6530-003 timer and the lengths with the 6530-002 timer (`CLKKT`, `CLKRDI`). Leave the last 1/16 s of each note silent so that repeated notes don't run together. Then `BRK`.

Try it: the first line of *Twinkle, Twinkle, Little Star*, in quarter notes: C C G G A A G (half note), F F E E D D C (half note).

36 Keypad organ

While a hex key is held down, play a note: key 0 is C4, and the keys go up the C major scale to D6 on key F. With no key down, stay silent.

Hint: `GETKEY` takes a noticeable part of a half period. Call it while the timer is already timing the current one.

Part 4 · The K-1008 Visible Memory

MTU's K-1008 Visible Memory is 8K of RAM that is also a picture: 320×200 dots on a monitor. The CPU uses it like any other memory, and the video circuit reads it between the CPU's accesses, so drawing never slows the program down. In Kimulator it is installed under **Machine > Expansion cards...**, at `$2000` by default. **View > Visible Memory display** (Ctrl+G) shows the screen.

Screen layout	
Size	320×200 dots, 1 bit each
Line y	40 bytes from <code>\$2000 + 40 × y</code>
Shown	<code>\$2000–\$3F3F</code> (8000 bytes)
Not shown	<code>\$3F40–\$3FFF</code> (192 bytes)
In a byte	bit 7 is the leftmost dot; 1 = lit

The dot at (x, y), with (0, 0) the top-left corner, is in

$$\text{byte} = \$2000 + 40 \times y + x \div 8 \quad \text{bit mask} = \$80 \gg (x \bmod 8)$$

x goes up to 319, so it needs 9 bits: keep it as a 16-bit number. $40 \times y$ reaches 7960, which is 16 bits too. The 6502 multiplies by 40 with shifts and adds: $40y = 8 \times (4y + y)$.

Set a dot with `ORA`, clear it with `AND` of the inverted mask, and flip it with `EOR`. Flipping twice restores what was there, which makes `EOR` the classic way to move something across a picture.

37 Paint the screen

Fill all 8K of the Visible Memory, `$2000–$3FFF`, with the byte at `$10`.

Try it: `$00` clears the screen, `$FF` makes it white and `$AA` draws thin vertical stripes. What does `$F0` draw?

Hint: Fill whole 256-byte pages with `(zp),Y` and step the pointer's high byte.

38 Checkerboard

Cover the screen with 8×8 dot squares, alternately black and white, black in the top-left corner.

Hint: Work line by line with a pointer to the start of the line, adding 40 each time. Whether byte c of line y is white depends on c and $y \div 8$.

39 Plot a dot

Write `PLOT`, which lights the dot at (XC, YC). XC is a 16-bit value (0–319) at `$30 / $31` and YC (0–199) is at `$32`. Use it to draw a frame around the edge of the screen and a diagonal from (0, 0) to (199, 199).

Hint: A table of the eight bit masks, `$80`, `$40`, ..., `$01`, indexed by $x \bmod 8$, is quicker than shifting.

40 Draw a line

Write `LINE`, which draws a straight line from (X0, Y0) to (X1, Y1), both ends included, in any direction. X0 and X1 are 16 bits and Y0 and Y1 are 8 bits. Draw a star of lines through the middle of the screen to test it.

Bresenham's algorithm needs only additions:

```
dx = |x1 - x0|,    sx = +1 if x0 < x1, else -1
dy = -|y1 - y0|,   sy = +1 if y0 < y1, else -1
err = dx + dy
loop: plot (x0, y0)
      if x0 = x1 and y0 = y1: done
      e2 = 2 * err
      if e2 ≥ dy: err = err + dy;  x0 = x0 + sx
      if e2 ≤ dx: err = err + dx;  y0 = y0 + sy
```

Hint: All these values fit easily in 16 signed bits. To test $e2 \geq dy$, subtract dy from e2 and look at bit 7 of the result's high byte.

41 Hello, world

Treat the screen as 40 columns × 25 rows of 8 × 8 dot character cells. Write `PUTCH`, which draws the character in A in cell (COL, ROW) and moves COL on by one. Write `PRINT`, which draws a zero-terminated string with `PUTCH`. Print “HELLO, WORLD!” in the middle of the screen and “KIM-1 6502” below it.

Hint: Cell (COL, ROW) starts at byte `$2000 + 320 × ROW + COL`, and its eight lines are 40 bytes apart. A full ASCII font doesn't fit in the KIM's RAM, so draw glyphs for just the characters you need, 5 × 7 dots in each 8 × 8 cell.

42 Scroll up

Move the whole picture up by one character row (8 dot lines), the way a terminal scrolls, and clear the bottom row.

Hint: Count the bytes to move: it comes out to a whole number of pages. Does the order you copy in matter?

43 Bouncing ball

Move an 8 × 8 ball around the screen, 2 dots at a time and 50 times a second, bouncing off all four edges. Draw it with `EOR`, so drawing it again in the same place erases it.

Hint: At position x, each 8-dot row of the ball's shape is shifted right by $x \bmod 8$ and spread over two bytes. Shift a 16-bit value, shape row : 0, right with `LSR / ROR`.

44 Sketch pad

A pen starts in the middle of a clear screen and leaves a trail of dots. While a key is held, the pen moves one dot every $1/50$ s: **9** up, **1** down, **4** left and **6** right. **0** clears the screen. The pen stops at the edges.

Going further: *Add a pen-up/pen-down key, or diagonal moves on 8, A, 0 and 2.*

Answers

These are the programs from `docs/workbook/answers`, without their header comments. Where an answer reuses a routine from an earlier one, the routine isn't printed again. The source file has it in full.

1 Hello, memory

`LDA #$42` loads the number itself. `STA $10` is a zero-page store: 2 bytes, 3 cycles.

```
.org $0200
start: lda #$42      ; '#' means immediate: A gets the number $42 itself
      sta $10       ; store A at address $0010 (zero page, 2-byte instruction)
      brk           ; back to the monitor
```

2 Copy cat

A store doesn't change the register, so one load serves all three stores.

```
.org $0200
start: lda $10       ; no '#': load the byte stored AT address $10
      sta $11       ; storing doesn't change A,
      sta $12       ; so one load is enough
      sta $13
      brk
```

3 Swap

With two registers there's somewhere to keep both values at once, so no temporary memory is needed.

```
.org $0200
start: ldx $10       ; X = old $10
      ldy $11       ; Y = old $11
      stx $11       ; each register holds one value, so no
      sty $10       ; temporary memory location is needed
      brk
```

4 Neighbours

Increments and decrements wrap around: `N = $FF` gives `$00` and `$FE`, and `N = $00` gives `$01` and `$FF`.

```
.org $0200
start: lda $10       ; A = N
      tax           ; X = N
      tay           ; Y = N
      inx           ; X = N+1 (wraps from $FF to $00)
      dey           ; Y = N-1 (wraps from $00 to $FF)
      stx $11
      sty $12
      brk
```

5 Change it in place

`INC` and `DEC` read, change and write memory in one instruction, without touching a register.

```
.org $0200
start: inc $10       ; INC and DEC work directly on memory:
```

```

inc $10      ; the CPU reads the byte, changes it and
dec $11      ; writes it back in one instruction
dec $11
dec $11
brk

```

6 Add two bytes

ADC always adds the carry. Whatever ran before your program, the monitor included, may have left it set, and then the sum would be one too big.

```

.org $0200
start: clc      ; ADC always adds the carry too, so clear it first
      lda $10
      adc $11    ; A = $10 + $11 + C
      sta $12
      brk

```

7 Subtract two bytes

For SBC the carry works the other way round: C = 1 means “nothing borrowed”. SEC first.

```

.org $0200
start: sec      ; for SBC the carry means "no borrow", so set it first
      lda $10
      sbc $11    ; A = $10 - $11 - (1 - C)
      sta $12
      brk

```

8 16-bit addition

One CLC at the start of the chain. The high bytes then add the carry left by the low bytes.

```

.org $0200
start: clc
      lda $10      ; low bytes first...
      adc $12
      sta $14
      lda $11      ; ...then the high bytes, adding the carry
      adc $13      ; that the low-byte addition left behind
      sta $15
      brk

```

9 16-bit subtraction

The same chain with SEC / SBC. A borrow out of the low byte clears the carry, which takes one more off the high byte.

```

.org $0200
start: sec
      lda $10      ; low bytes first
      sbc $12
      sta $14
      lda $11      ; the borrow (carry clear) flows into the high byte
      sbc $13
      sta $15
      brk

```

10 Times ten

Five additions. With the shifts from Part 2 there is a shorter way: $10N = 2 \times (4N + N)$ is three `ASL` s and one addition.

```
.org $0200
start: lda $10
      clc
      adc $10      ; A = 2N
      sta $12      ; keep 2N in a scratch location
      clc
      adc $12      ; A = 4N
      clc
      adc $12      ; A = 6N
      clc
      adc $12      ; A = 8N
      clc
      adc $12      ; A = 10N (at most 250, so no carry ever)
      sta $11
      brk
```

11 Increment a 24-bit number

The same carry chain as problem 8. `ADC #0` adds only the carry, which is 1 exactly when the byte below wrapped from `$FF` to `$00`.

```
.org $0200
start: clc
      lda $10
      adc #1      ; add 1 to the low byte...
      sta $10
      lda $11
      adc #0      ; ...and only the carry to the others:
      sta $11      ; it is 1 exactly when the byte below
      lda $12      ; wrapped from $FF to $00
      adc #0
      sta $12
      brk
```

12 Negate a 16-bit number

$0 - x$ as a 16-bit subtraction. `$8000` (-32768) has no positive partner in 16 bits, so it comes out as itself.

```
.org $0200
start: sec
      lda #0
      sbc $10      ; 0 - low byte
      sta $12
      lda #0
      sbc $11      ; 0 - high byte - borrow
      sta $13
      brk
```

13 The bigger of two

After `CMP`, `C = 1` means $A \geq$ the operand. Only when that fails does the program load the other value.

```
.org $0200
start:  lda $10
        cmp $11      ; compare A with $11: C = 1 when A >= $11
        bcs @done    ; $10 is already the larger (or equal)
        lda $11      ; otherwise $11 is larger
@done:  sta $12
        brk
```

14 Counting tables

The first loop stops when `CPX` finds `X = $40`. The second counts `X` down from `$3F` and stops when `DEX` wraps 0 to `$FF`, the first value with bit 7 set, so `BPL` falls through.

```
.org $0200
start:  ldx #0
@up:    txa
        sta $0300,x  ; absolute,X: address $0300 + X
        inx
        cpx #$40     ; stop after X = $3F
        bne @up

        ldx #$3F     ; a second loop counting down:
        ldy #0       ; X is the value, Y the offset
@down:  txa
        sta $0340,y
        iny
        dex
        bpl @down    ; loop while X >= 0 (bit 7 clear): DEX of 0 gives $FF
        brk
```

15 Sum a table

An 8-bit addition into the low byte, and an `INC` of the high byte whenever it carries.

```
.org $0200
start:  lda #0
        sta $10
        sta $11
        ldx #0
@loop:  clc
        lda $10
        adc $0300,x  ; add the next byte to the low byte of the sum
        sta $10
        bcc @next    ; no carry: high byte unchanged
        inc $11      ; carry: bump the high byte
@next:  inx
        cpx #32
        bne @loop
        brk
```

16 Smallest and largest

Starting from the first element avoids having to choose “infinitely large” and “infinitely small” start values.

```
.org $0200
start:  lda $0301      ; the first element is both min and max so far
        sta $10
        sta $11
        ldx #1        ; X = number of elements examined
@loop:  cpx $0300
        beq @done     ; examined all N
        inx
        lda $0300,x   ; element X (they start at $0301)
        cmp $10
        bcs @notmin   ; A >= min
        sta $10
@notmin:
        cmp $11
        bcc @loop     ; A < max
        sta $11
        jmp @loop
@done:  brk
```

17 Masks

AND clears, ORA sets and EOR flips the bits that are 1 in the mask. BIT copies bit 6 of memory into V without changing A.

```
.org $0200
start:  lda $10
        and #%00001111 ; keep only bits 0-3
        sta $11
        lda $10
        ora #%10000001 ; force bits 7 and 0 on
        sta $12
        lda $10
        eor #%11111111 ; flip every bit
        sta $13

        ldx #0
        bit $10        ; BIT copies bit 7 to N and bit 6 to V
        bvc @clear     ; (and leaves A alone)
        inx
@clear: stx $14
        brk
```

18 Count the ones

LSR moves bit 0 into the carry and sets Z when nothing is left in A, so the loop never runs more times than it needs to. INX changes Z, which is why A is tested again after it.

```
.org $0200
start:  lda $10
        ldx #0        ; X counts the ones
        cmp #0
@loop:  beq @done     ; no 1 bits left in A
        lsr a        ; shift bit 0 into the carry; Z tells if A is now 0
        bcc @loop
        inx
        jmp @loop
```

```

    inx                ; INX changes Z, so test A again
    cmp #0
    jmp @loop
@done: stx $11
    brk

```

19 Multiply

The multiplier is shifted right to look at one bit at a time, and the multiplicand is doubled for the next bit position. The loop stops when no 1 bits are left in the multiplier.

```

    .org $0200
MCAND = $14                ; 16-bit copy of the multiplicand we can shift
start: lda $10
    sta MCAND
    lda #0
    sta MCAND+1
    sta $12
    sta $13
    lda $11                ; A = multiplier, used up one bit at a time
@loop: lsr a                ; lowest bit of the multiplier into C
    tax                    ; save the rest (the addition needs A)
    bcc @skip
    clc
    lda $12
    adc MCAND
    sta $12
    lda $13
    adc MCAND+1
    sta $13
@skip: asl MCAND            ; double the multiplicand:
    rol MCAND+1            ; ASL the low byte, ROL its carry into the high byte
    tax                    ; the multiplier's remaining bits
    bne @loop              ; stop when none are left
    brk

```

20 Byte to hex text

PHA / PLA keep the byte while the high digit is converted. In NIBBLE, the carry left by CMP #10 is part of the arithmetic: for 10–15 it is set, so ADC #6 adds 7.

```

    .org $0200
start: lda $10
    pha                ; keep the byte for the low digit
    lsr a                ; move the high nibble down
    lsr a
    lsr a
    lsr a
    jsr NIBBLE
    sta $11
    pla                ; the original byte again
    and #$0F
    jsr NIBBLE
    sta $12
    brk

; A = 0-15 -> A = '0'-'9' or 'A'-'F'
NIBBLE: cmp #10

```

```

        bcc @digit      ; 0-9 (carry clear)
        adc #6          ; 10-15: carry is set, so this adds 7, the gap from '9'+1 to 'A'
@digit: adc #'0'        ; carry is clear on both paths
        rts

```

21 Block copy

Both pointers use the same Y. If the destination overlaps the source from above, copying upwards overwrites bytes before they're copied. Copy from the top down instead, as `memmove` does.

```

        .org $0200
SRC      = $10
DST      = $12
COUNT   = $14
start:   ldy #0
@loop:   lda (SRC),y    ; indirect indexed: address = (word at SRC) + Y
        sta (DST),y
        iny
        cpy COUNT
        bne @loop
        brk

```

22 SHOUT

Lowercase and uppercase ASCII letters differ only in bit 5, but only letters may be changed, hence the range check. When the loop ends, Y has counted the characters.

```

        .org $0200
STR      = $10
start:   ldy #0
@loop:   lda (STR),y
        beq @end      ; the terminating zero
        cmp #'a'
        bcc @next     ; below 'a'
        cmp #'z'+1
        bcs @next     ; above 'z'
        and #%11011111 ; lowercase and uppercase differ only in bit 5
        sta (STR),y
@next:   iny
        bne @loop
@end:    sty $12
        brk

```

23 Decimal addition

Exactly problem 8 in decimal mode. `CLD` as soon as the BCD work is done: anything that runs later in decimal mode, a monitor routine included, gets wrong results.

```

        .org $0200
start:   sed          ; decimal mode: ADC and SBC work in BCD
        clc
        lda $10
        adc $12
        sta $14
        lda $11
        adc $13
        sta $15

```

```

cld                ; always leave decimal mode when done!
lda #0
rol a              ; the final carry into bit 0
sta $16
brk

```

24 Binary to decimal

ASL takes the next bit into the carry. ADC of a value to itself then gives $2 \times \text{value} + \text{bit}$, and in decimal mode that stays BCD. The carry out of the tens is added into the hundreds.

```

.org $0200
start: lda $10
      sta $13          ; working copy we shift bits out of
      lda #0
      sta $11
      sta $12
      ldx #8
      sed
@loop: asl $13          ; next bit (from the top) into C
      lda $12
      adc $12          ; tens and units: 2 x value + bit, in BCD
      sta $12
      lda $11
      adc $11          ; hundreds: 2 x value + carry out of the tens
      sta $11
      dex
      bne @loop
      cld
      brk

```

25 Sort

For the “going further” version, keep the number of pairs to compare in memory and reduce it by one after each pass. The worst case, a reversed list, needs $N - 1$ passes.

```

.org $0200
LIST    = $0301        ; the first element
SWAPPED = $10
start:
@pass:  lda #0
      sta SWAPPED
      ldx #0
@inner: inx              ; compare elements X-1 and X (counting from 0)
      cpx $0300
      beq @endpass      ; X = N: every pair has been compared
      lda LIST-1,x
      cmp LIST,x
      bcc @inner        ; already in order (A < next)
      beq @inner        ; equal: leave them
      tay              ; swap them, using Y as the spare register
      lda LIST,x
      sta LIST-1,x
      tya
      sta LIST,x
      inc SWAPPED
      jmp @inner
@endpass:

```

```

lda SWAPPED
bne @pass      ; a pass with no swaps means the list is sorted
brk

```

26 Say something

SCANDS lights each digit once and returns. The display only looks steady because the loop calls it again straight away.

```

.org $0200
start: lda #$C0
      sta POINTH      ; $FB: the two leftmost digits
      lda #$DE
      sta POINTL      ; $FA: the next two
      lda #$42
      sta INH         ; $F9: the two data digits
@loop: jsr SCANDS      ; one pass over all six digits (about 4 ms)
      jmp @loop

```

27 Calculator entry

The 24-bit shift moves the digits left. Four shifts clear the low digit, and the new key is ORA'd in. A real keypad also bounces for a few milliseconds, but the 4 ms that SCANDS takes covers that.

```

.org $0200
start: lda #0
      sta POINTH
      sta POINTL
      sta INH
@wait: jsr SCANDS      ; show the digits; returns Z = 1 while no key is down
      beq @wait
      jsr GETKEY       ; A = the key's code: 0-F, $10-$14 for AD DA + GO PC
      cmp #$10
      bcs @release     ; not a hex key (or $15: released too soon)
      ldx #4           ; shift POINTH:POINTL:INH left by one digit (4 bits)
@shift: asl INH
      rol POINTL
      rol POINTH
      dex
      bne @shift
      ora INH          ; the new digit goes into the low 4 bits, now 0
      sta INH
@release: jsr SCANDS   ; keep the display lit until the key is let go
      bne @release
      jmp @wait

```

28 Tally counter

A BCD count shows in decimal on the hex display. CLD before returning to SCANDS.

```

.org $0200
start: lda #0
      sta POINTH
      sta POINTL
      sta INH
@wait: jsr SCANDS
      beq @wait      ; no key down

```

```

jsr GETKEY
cmp #$15
bcs @release    ; no valid key after all
sta INH
sed            ; count in BCD so the display shows decimal
clc
lda POINTL
adc #1
sta POINTL
lda POINTH
adc #0          ; carry from 99 -> 00 in the low byte
sta POINTH
cld            ; SCANDS must run in binary mode
@release:
jsr SCANDS
bne @release
jmp @wait

```

29 HELLO

This is what SCANDS does inside: select a digit, drive its segments, wait, move on. ASL of a number below 128 leaves the carry clear, so the ADC #8 needs no CLC.

```

.org $0200
start: lda #$7F
      sta PADD    ; $1741: PA0-PA6 are outputs
      lda #$1E
      sta PBDD    ; $1743: PB1-PB4 are outputs
@frame: ldx #0    ; X = digit number
@digit: lda #0
      sta SAD     ; blank the segments while switching digits
      txa
      asl a       ; X x 2 ...
      adc #8      ; ... + 8 = (X + 4) x 2 (ASL left the carry clear)
      sta SBD     ; select digit X
      lda MSG,x
      sta SAD     ; light its segments
      ldy #100    ; keep it lit for about 0.5 ms
@delay: dey
      bne @delay
      inx
      cpx #6
      bne @digit
      jmp @frame

;           H   E   L   L   O   (blank)
MSG: .byte $76, $79, $38, $38, $3F, $00

```

30 Marquee

The timer runs by itself while the program scans the digits, and one BIT per pass is enough to notice when it runs out. Y wraps at the end of the message, so it scrolls round forever.

```

.org $0200
TIMER1024 = $1707    ; write: start the timer counting every 1024 cycles
TIMEOUT   = $1707    ; read: bit 7 is set once the timer has run out
QUARTER   = 243      ; the flag rises after (243 + 1) x 1024 us = 0.2499 s
POS       = $10      ; message position shown in the leftmost digit

```

```

COUNT    = $11                ; delay counter

start:     lda #$7F
          sta PADD
          lda #$1E
          sta PBDD
          lda #0
          sta POS
          lda #QUARTER
          sta TIMER1024

@frame:    ldx #0                ; X = digit 0-5
          ldy POS                ; Y = message index for that digit
@digit:    lda #0
          sta SAD
          txa
          asl a
          adc #8
          sta SBD
          lda MSG,y
          sta SAD
          lda #60                ; about 0.5 ms
          sta COUNT
@delay:    dec COUNT
          bne @delay
          iny                    ; next character, wrapping at the end of the message
          cpy #LEN
          bne @nowrap
          ldy #0
@nowrap:
          inx
          cpx #6
          bne @digit

          bit TIMEOUT            ; BIT puts bit 7 of the flag register in N
          bpl @frame            ; not time to move yet
          lda #QUARTER
          sta TIMER1024          ; start the next quarter second (also clears the flag)
          inc POS
          lda POS
          cmp #LEN
          bne @frame
          lda #0
          sta POS
          jmp @frame

;
;           H   E   L   L   O           6   5   0   2
MSG:        .byte $76, $79, $38, $38, $3F, $00, $7D, $6D, $3F, $5B, $00, $00
LEN         = * - MSG

```

31 Read the keypad yourself

EOR #\$FF turns “pulled low” into a 1, so any nonzero value means a key is down. The shifts then find the column, adding one to the code for each column passed.

```

          .org $0200
start:     lda #0
          sta POINTH
          sta POINTL

```

```

    sta INH
@loop: jsr READKEY
    cmp #$15
    beq @show      ; nothing pressed: keep the old code
    sta INH
@show:  jsr SCANDS
    jmp @loop

READKEY:
    lda #$00
    sta PADD      ; PA0-PA7 are inputs
    lda #$1E
    sta PBDD      ; PB1-PB4 drive the 74145
    ldx #0        ; X = row
    ldy #0        ; Y = code of the first key in the row
@row:   txa
    asl a
    sta SBD      ; pull row X low
    lda SAD
    ora #$80      ; PA7 is the TTY input, not a key
    eor $FF       ; now a 1 marks a pressed key
    bne @found
    tya
    clc
    adc #7        ; the next row starts 7 codes later
    tay
    inx
    cpx #3
    bne @row
    lda #$15      ; no key
    rts

@found: asl a      ; drop bit 7 (always 0 here)
@col:   asl a      ; next column into C: PA6 first, down to PA0
    bcs @done
    iny          ; not this one: the code is one higher
    bne @col     ; always taken
@done:  tya
    rts

```

32 Egg timer

SCANDS takes about 4 ms, so the flag can be up to 4 ms old when the program sees it. Restarting the timer right away limits that to a small error per quarter second. If the restart comes later in the loop, every delay adds up: that is why the “going further” version runs slow.

```

.org $0200
TIMER1024 = $1707      ; write: start the timer, one count per 1024 cycles
TIMEOUT   = $1707      ; read: bit 7 set when the timer has run out
QUARTER   = 243        ; the flag rises after (243 + 1) x 1024 us = 0.2499 s
QUARTERS  = $10        ; quarter seconds left in this second

start:  lda MINS
    sta POINTH
    lda SECS
    sta POINTL
    lda #0
    sta INH

```

```

lda #4
sta QUARTERS
lda #QUARTER
sta TIMER1024

@tick: jsr SCANDS      ; about 4 ms
bit TIMEOUT
bpl @tick
lda #QUARTER      ; restart at once, so the time the rest of the
sta TIMER1024      ; loop takes doesn't add up
dec QUARTERS
bne @tick
lda #4
sta QUARTERS

sed              ; one second less, in BCD
sec
lda POINTL
sbc #1
bcs @store      ; no borrow: seconds were 01-59
lda #$59        ; 00 -> 59 and one minute less
sta POINTL
lda POINTH
sbc #0          ; carry is still clear, so this subtracts 1
sta POINTH
jmp @check
@store: sta POINTL
@check: cld
lda POINTL
ora POINTH
bne @tick      ; not 0000 yet

lda #$EE      ; done
sta INH
@done: jsr SCANDS
jmp @done

MINS: .byte $03      ; change these to set the time
SECS: .byte $00

```

33 Interrupt-driven stopwatch

The interrupt happens within a few microseconds of the time-out, however busy the main loop is. The handler can safely use decimal mode: the interrupt saved P, with the main program's D flag, and RTI restores it. $155 + 1$ steps of $64 \mu\text{s}$ is 9.984 ms . Add the handler's start-up time, and the watch is within about 0.1 %.

```

.org $0200
TIMER64I = $170E      ; write: start the /64 timer with its interrupt enabled
TICK     = 155         ; the interrupt comes  $(155 + 1) \times 64 \mu\text{s} = 9.984 \text{ ms}$  later
RUNNING  = $10         ; 0 = stopped, 1 = running

start: sei            ; no interrupts while we set things up
lda #<ISR      ; the monitor's IRQ handler jumps through IRQV ($17FE)
sta IRQV
lda #>ISR
sta IRQV+1
lda #0
sta POINTH

```

```

    sta POINTL
    sta INH
    sta RUNNING
    lda #TICK
    sta TIMER64I
    cli                ; interrupts on

@wait: jsr SCANDS
       beq @wait      ; no key down
       lda RUNNING
       eor #1         ; start <-> stop
       sta RUNNING

@release:
       jsr SCANDS
       bne @release
       jmp @wait

; Runs every 10 ms, in the middle of whatever the main loop is doing.
; The CPU has already saved PC and P; we must save any register we change.
ISR:   pha
       lda #TICK
       sta TIMER64I   ; restart the timer; writing it also clears the interrupt
       lda RUNNING
       beq @out
       sed            ; RTI restores the main program's D flag
       clc
       lda INH
       adc #1         ; hundredths
       sta INH
       bcc @out       ; no carry: 00-99 still
       lda POINTL
       adc #0         ; carry is set: one more second
       cmp #$60
       bne @sec
       lda #0
@sec:   sta POINTL
       bne @out       ; not a new minute
       clc
       lda POINTH
       adc #1
       sta POINTH
@out:   pla
       rti

```

34 Concert A

1/880 s is 1136 μ s, 142 steps of the $\div 8$ timer. Take one off because the flag rises after $N + 1$ steps, and two more for the time the loop takes to notice the flag and restart.

```

.org $0200
APORTA = $1700      ; application port A data
APORTAD = $1701     ; its data direction register (1 = output)
TIMER8 = $1705      ; write: start the timer, one count per 8 cycles
TIMEOUT = $1707     ; read: bit 7 set when the timer has run out
HALF = 139          ; half a period, in 8 us steps

start: lda #$01
       sta APORTAD  ; PA0 is an output
@loop: lda #HALF

```

```

    sta TIMER8      ; start timing the next half period first...
    lda APORTA
    eor #$01        ; ...then flip PA0
    sta APORTA
@wait: bit TIMEOUT
    bpl @wait
    jmp @loop

```

35 Play a tune

One loop watches both timers: the pitch timer flips the pin, and the length timer counts sixteenths. This loop takes longer to go round than problem 34's, so the pitch values are a little smaller.

```

.org $0200
APORTA = $1700
APORTAD = $1701
TIMER8 = $1705      ; 6530-003: start the /8 timer
TIMEOUT = $1707      ; 6530-003: bit 7 = timer ran out
; CLKKT ($1747) and CLKRDI ($1747) are the monitor's names for the
; 6530-002's /1024 timer (write) and its flag (read).
UNIT = 60           ; (60 + 1) x 1024 us = 62.5 ms = 1/16 s

; Half periods in 8 us steps, less 1 (the timer runs N + 1 steps)
; and less 5 for the loop's own time (it watches two timers, so it is
; slower than problem 34's).
C4 = 233
D4 = 207
E4 = 184
F4 = 173
G4 = 153
A4 = 136

PITCH = $10
LEFT = $11          ; units left in this note

start: lda #$01
    sta APORTAD
    ldx #0           ; X = offset of the current note in TUNE
@note: lda TUNE+1,x
    beq @end         ; Length 0: the end
    sta LEFT
    lda TUNE,x
    sta PITCH
    inx
    inx
    lda #UNIT
    sta CLKKT        ; start the first 1/16 s
    lda PITCH
    sta TIMER8

@play: bit TIMEOUT    ; half period over?
    bpl @unit
    lda PITCH
    sta TIMER8        ; time the next one
    beq @unit         ; a rest: don't flip the pin
    ldy LEFT
    cpy #1
    beq @unit         ; the last unit of a note is silent
    lda APORTA

```

```

    eor #$01
    sta APORTA
@unit: bit CLKRDI      ; 1/16 s over?
    bpl @play
    lda #UNIT
    sta CLKKT
    dec LEFT
    bne @play
    jmp @note
@end:   brk

; Twinkle, twinkle, little star (first line)
TUNE:   .byte C4,4, C4,4, G4,4, G4,4, A4,4, A4,4, G4,8
        .byte F4,4, F4,4, E4,4, E4,4, D4,4, D4,4, C4,8
        .byte 0,0

```

36 Keypad organ

The timer is started and the pin flipped before `GETKEY` runs, so reading the keypad doesn't change the pitch. Even D6's half period (0.43 ms) is longer than `GETKEY` takes. `GETKEY` relies on `PADD = 0`, so the program sets it, because no `SCANDS` runs here.

```

    .org $0200
APORTA = $1700
APORTAD = $1701
TIMER8 = $1705
TIMEOUT = $1707
NOTE = $10      ; half period for the key that is down, 0 = none

start: lda #$01
    sta APORTAD
    lda #$00
    sta PADD      ; keypad columns are inputs (GETKEY expects SCANDS to have done this)
    lda #0
    sta NOTE
@loop: lda NOTE
    beq @key      ; silent: just read the keypad
    sta TIMER8    ; time this half period...
    lda APORTA
    eor #$01
    sta APORTA
@key:   jsr GETKEY ; ...while we read the keypad
    cmp #$10
    bcc @hex
    lda #0        ; no key, or not a hex key
    beq @store
@hex:   tax
    lda NOTES,x
@store: sta NOTE
    beq @loop     ; no note: don't wait for the timer
@wait:  bit TIMEOUT
    bpl @wait
    jmp @loop

; Half periods in 8 us steps, Less 3 as in problem 35: C4 D4 E4 F4 G4 A4 B4 C5 ... D6
NOTES: .byte 236, 210, 187, 176, 156, 139, 124, 116
        .byte 103, 92, 86, 77, 68, 60, 57, 50

```

37 Paint the screen

32 pages of 256 bytes. `$F0` gives vertical stripes 4 dots wide.

```
.org $0200
SCREEN = $2000
PTR = $20      ; pointer to the page being filled

start: lda #<SCREEN    ; always 0: we fill whole pages
      sta PTR
      lda #>SCREEN
      sta PTR+1
      ldx #32          ; 32 pages of 256 bytes = 8K
      lda $10
      ldy #0
@page: sta (PTR),y      ; fill one page, Y = 0, 1, ..., 255
      iny
      bne @page
      inc PTR+1        ; next page
      dex
      bne @page
      brk
```

38 Checkerboard

Looking the byte up in a two-entry table avoids a branch. The line pointer steps by 40 with a 16-bit addition.

```
.org $0200
SCREEN = $2000
PTR = $20      ; start of the current line
LINE = $22      ; 0-199
ROWBIT = $23    ; bit 0 of (LINE / 8)

start: lda #<SCREEN
      sta PTR
      lda #>SCREEN
      sta PTR+1
      lda #0
      sta LINE

@line: lda LINE
      lsr a      ; LINE / 8
      lsr a
      lsr a
      and #1
      sta ROWBIT
      ldy #0      ; Y = byte within the line, 0-39
@byte: tya
      eor ROWBIT
      and #1
      tax
      lda PATTERN,x ; 0 -> black byte, 1 -> white byte
      sta (PTR),y
      iny
      cpy #40
      bne @byte

      clc      ; PTR = PTR + 40: the next line
      lda PTR
```

```

    adc #40
    sta PTR
    bcc @same
    inc PTR+1
@same: inc LINE
    lda LINE
    cmp #200
    bne @line
    brk

PATTERN: .byte $00, $FF

```

39 Plot a dot

$40 \times y$ is computed as $((4y + y) \times 8)$. $x \div 8$ shifts bit 8 of x into the result with `ROR`. The mask table turns $x \bmod 8$ into the dot's bit.

```

    .org $0200
SCREEN = $2000
XC     = $30      ; 16-bit x
YC     = $32      ; y
ADDR   = $33      ; 16-bit address PLOT works out

start: lda #0      ; top and bottom edges: x = 0-319
    sta XC
    sta XC+1
@hor:  lda #0
    sta YC
    jsr PLOT
    lda #199
    sta YC
    jsr PLOT
    inc XC          ; 16-bit increment
    bne @nocarry
    inc XC+1
@nocarry:
    lda XC+1        ; stop at x = 320 ($0140)
    cmp #>320
    bne @hor
    lda XC
    cmp #<320
    bne @hor

    lda #0          ; left and right edges: y = 0-199
    sta YC
@ver:  lda #0
    sta XC
    sta XC+1
    jsr PLOT
    lda #<319
    sta XC
    lda #>319
    sta XC+1
    jsr PLOT
    inc YC
    lda YC
    cmp #200
    bne @ver

```

```

    lda #0          ; diagonal: (0,0), (1,1), ..., (199,199)
    sta XC+1
    sta YC
@diag: lda YC
    sta XC
    jsr PLOT
    inc YC
    lda YC
    cmp #200
    bne @diag
    brk

; Light the dot at (XC, YC). Changes A, X and Y.
PLOT:  lda #0
    sta ADDR+1
    lda YC
    asl a          ; YC x 2 (the high bits go into ADDR+1 as we go)
    rol ADDR+1
    asl a          ; YC x 4
    rol ADDR+1
    clc
    adc YC         ; YC x 5
    bcc @x5
    inc ADDR+1
@x5:  asl a        ; YC x 10
    rol ADDR+1
    asl a        ; YC x 20
    rol ADDR+1
    asl a        ; YC x 40
    rol ADDR+1
    sta ADDR

    lda XC+1      ; XC / 8 (fits in a byte: at most 39)
    lsr a         ; bit 8 of XC into the carry...
    lda XC
    ror a         ; ...and from there into bit 7
    lsr a
    lsr a
    clc
    adc ADDR      ; ADDR = SCREEN + YC x 40 + XC / 8
    sta ADDR
    lda ADDR+1
    adc #>SCREEN
    sta ADDR+1

    lda XC
    and #7
    tax
    lda BITS,x    ; the dot's bit in its byte
    ldy #0
    ora (ADDR),y  ; set it, leaving the other 7 dots alone
    sta (ADDR),y
    rts

```

```

BITS:  .byte $80, $40, $20, $10, $08, $04, $02, $01

```

40 Draw a line

sx is kept as a 16-bit `$0001` or `$FFFF`, so stepping x is an ordinary 16-bit addition. Each comparison is a subtraction whose result is thrown away, all but the sign of its high byte.

```
.org $0200
SCREEN = $2000
XC      = $30      ; PLOT's inputs, as in problem 39
YC      = $32
ADDR    = $33
X0      = $40      ; 16 bits; LINE moves it to X1
Y0      = $42      ; LINE moves it to Y1
X1      = $43      ; 16 bits
Y1      = $45
DX      = $46      ; 16 bits, >= 0
DY      = $48      ; 16 bits, <= 0
SX      = $4A      ; 16 bits: $0001 or $FFFF
SY      = $4C      ; 8 bits: $01 or $FF
ERR     = $4D      ; 16 bits
E2      = $4F      ; 16 bits

start:  ldx #0      ; each ENDS entry: x0, y0, x1, y1 (x as words)
@next:  lda ENDS,x
        sta X0
        lda ENDS+1,x
        sta X0+1
        lda ENDS+2,x
        sta Y0
        lda ENDS+3,x
        sta X1
        lda ENDS+4,x
        sta X1+1
        lda ENDS+5,x
        sta Y1
        txa
        pha      ; LINE changes X
        jsr LINE
        pla
        clc
        adc #6
        tax
        cpx #ENDSLEN
        bne @next
        brk

ENDS:   .word 0
        .byte 0
        .word 319
        .byte 199
        .word 319
        .byte 0
        .word 0
        .byte 199
        .word 160
        .byte 0
        .word 160
        .byte 199
        .word 0
        .byte 100
```

```

.word 319
.byte 100
.word 100
.byte 20
.word 220
.byte 180
.word 220
.byte 20
.word 100
.byte 180
ENDSLEN = * - ENDS

LINE:  ldx #$00          ; dx = x1 - x0, sx = +1 ...
       sec
       lda X1
       sbc X0
       sta DX
       lda X1+1
       sbc X0+1
       sta DX+1
       bpl @dxpos
       sec              ; ... or, if that's negative, dx = -dx, sx = -1
       lda #0
       sbc DX
       sta DX
       lda #0
       sbc DX+1
       sta DX+1
       ldx #$FF
@dxpos: stx SX+1        ; sx = $0001 or $FFFF
       txa
       ora #$01
       sta SX

       ldx #$01        ; |y1 - y0| and sy
       sec
       lda Y1
       sbc Y0
       bcs @dypos      ; no borrow: y1 >= y0
       eor #$FF        ; negate: y0 - y1
       adc #1          ; (carry is clear here)
       ldx #$FF
@dypos: stx SY
       sta DY          ; dy = -|y1 - y0|
       sec
       lda #0
       sbc DY
       sta DY
       lda #0
       sbc #0
       sta DY+1

       clc             ; err = dx + dy
       lda DX
       adc DY
       sta ERR
       lda DX+1
       adc DY+1
       sta ERR+1

```

```

@loop:  lda X0          ; plot (x0, y0)
        sta XC
        lda X0+1
        sta XC+1
        lda Y0
        sta YC
        jsr PLOT

        lda X0          ; at the end?
        cmp X1
        bne @go
        lda X0+1
        cmp X1+1
        bne @go
        lda Y0
        cmp Y1
        bne @go
        rts

@go:    lda ERR          ; e2 = 2 x err
        asl a
        sta E2
        lda ERR+1
        rol a
        sta E2+1

        sec              ; e2 >= dy <=> e2 - dy >= 0
        lda E2
        sbc DY
        lda E2+1
        sbc DY+1
        bmi @noxstep
        clc              ; err = err + dy
        lda ERR
        adc DY
        sta ERR
        lda ERR+1
        adc DY+1
        sta ERR+1

        clc              ; x0 = x0 + sx
        lda X0
        adc SX
        sta X0
        lda X0+1
        adc SX+1
        sta X0+1

@noxstep:
        sec              ; e2 <= dx <=> dx - e2 >= 0
        lda DX
        sbc E2
        lda DX+1
        sbc E2+1
        bmi @loop
        clc              ; err = err + dx
        lda ERR
        adc DX
        sta ERR
        lda ERR+1
        adc DX+1

```

```

sta ERR+1
clc           ; y0 = y0 + sy
lda Y0
adc SY
sta Y0
jmp @loop

```

; PLOT (problem 39): not repeated here

41 Hello, world

PUTCH finds the character in `CHARS`, and its position there $\times 8$ is where its glyph starts in `FONT`. The glyphs use bits 6–2 of each byte, which leaves a column of space on each side.

```

.org $0200
SCREEN = $2000
STR    = $10           ; string pointer
SIDX   = $12           ; index into the string
COL    = $13
ROW    = $14
ADDR   = $15           ; 16 bits
LINES  = $17           ; dot lines left to draw

start: lda #10
      sta ROW
      lda #13
      sta COL
      lda #<HELLO
      sta STR
      lda #>HELLO
      sta STR+1
      jsr PRINT
      lda #12
      sta ROW
      lda #15
      sta COL
      lda #<KIM
      sta STR
      lda #>KIM
      sta STR+1
      jsr PRINT
      brk

HELLO: .byte "HELLO, WORLD!", 0
KIM:   .byte "KIM-1 6502", 0

PRINT: lda #0
      sta SIDX
@loop: ldy SIDX
      lda (STR),y
      beq @done
      jsr PUTCH
      inc SIDX
      bne @loop
@done: rts

PUTCH: ldx #NCHARS-1   ; find the character in CHARS
@find: cmp CHARS,x
      beq @found

```

```

dex
bne @find      ; not found: X = 0, the space
@found: txa    ; X = index x 8: where its glyph starts in FONT
        asl a
        asl a
        asl a
        tax

        lda #0      ; ADDR = ROW x 64 ...
        sta ADDR+1
        lda ROW
        ldy #6
@x64:   asl a
        rol ADDR+1
        dey
        bne @x64
        clc          ; ... + COL
        adc COL
        sta ADDR
        lda ADDR+1   ; ... + ROW x 256 + SCREEN
        adc ROW
        adc #>SCREEN ; (no carry: the sum is under $40)
        sta ADDR+1

        lda #8
        sta LINES
        ldy #0
@line:  lda FONT,x
        sta (ADDR),y
        inx
        clc          ; next dot line: 40 bytes on
        lda ADDR
        adc #40
        sta ADDR
        bcc @same
        inc ADDR+1
@same:  dec LINES
        bne @line
        inc COL
        rts

CHARS:  .byte " !,-01256DEHIKLMORW"
NCHARS = * - CHARS
FONT:   .byte $00, $00, $00, $00, $00, $00, $00, $00 ; ' '
        .byte $10, $10, $10, $10, $10, $00, $10, $00 ; '!'
        .byte $00, $00, $00, $00, $00, $30, $10, $20 ; ','
        .byte $00, $00, $00, $7C, $00, $00, $00, $00 ; '-'
        .byte $38, $44, $4C, $54, $64, $44, $38, $00 ; '0'
        .byte $10, $30, $10, $10, $10, $10, $38, $00 ; '1'
        .byte $38, $44, $04, $08, $10, $20, $7C, $00 ; '2'
        .byte $7C, $40, $78, $04, $04, $44, $38, $00 ; '5'
        .byte $18, $20, $40, $78, $44, $44, $38, $00 ; '6'
        .byte $78, $44, $44, $44, $44, $44, $78, $00 ; 'D'
        .byte $7C, $40, $40, $78, $40, $40, $7C, $00 ; 'E'
        .byte $44, $44, $44, $7C, $44, $44, $44, $00 ; 'H'
        .byte $38, $10, $10, $10, $10, $10, $38, $00 ; 'I'
        .byte $44, $48, $50, $60, $50, $48, $44, $00 ; 'K'
        .byte $40, $40, $40, $40, $40, $7C, $00 ; 'L'
        .byte $44, $6C, $54, $54, $44, $44, $44, $00 ; 'M'
        .byte $38, $44, $44, $44, $44, $44, $38, $00 ; 'O'

```

```
.byte $78, $44, $44, $78, $50, $48, $44, $00 ; 'R'
.byte $44, $44, $44, $54, $54, $54, $28, $00 ; 'W'
```

42 Scroll up

7680 bytes is exactly 30 pages. Copying upwards is safe because the destination is below the source. Each byte is read before anything is written over it.

```
.org $0200
SCREEN = $2000
SRC = $20
DST = $22

start: lda #<(SCREEN+320)
      sta SRC
      lda #>(SCREEN+320)
      sta SRC+1
      lda #<SCREEN
      sta DST
      lda #>SCREEN
      sta DST+1
      ldx #30 ; pages to move
      ldy #0
@copy: lda (SRC),y
      sta (DST),y
      iny
      bne @copy
      inc SRC+1 ; both pointers move on a page
      inc DST+1
      dex
      bne @copy

      lda #0 ; DST is now $3E00: clear 256 + 64 bytes
@clear: sta (DST),y ; Y is 0 again here
      iny
      bne @clear
      inc DST+1
@clear2:
      sta (DST),y
      iny
      cpy #64
      bne @clear2
      brk
```

43 Bouncing ball

The ball is erased by drawing it again, then moved and drawn once more, all right after the frame timer runs out, so it's never missing from the picture for long. The bounces are exact because every position is even.

```
.org $0200
SCREEN = $2000
TIMER1024 = $1707 ; 6530-003: write starts the /1024 timer
TIMEOUT = $1707 ; read: bit 7 = timer ran out
FRAMETIME = 19 ; (19 + 1) x 1024 us = 20.5 ms
MAXX = 320 - 8
MAXY = 200 - 8
BX = $30 ; 16-bit x of the ball's left edge, even
BY = $32 ; y of its top line, even
```

```

DX      = $33          ; 16 bits: +2 or -2
DY      = $35          ; +2 or -2
ADDR    = $36          ; 16 bits
SHIFT   = $38          ; x mod 8
LEFT    = $39          ; a shifted shape row: left byte...
RIGHT   = $3A          ; ...and right byte

```

```

start:  lda #100
        sta BX
        lda #0
        sta BX+1
        sta DX+1
        lda #50
        sta BY
        lda #2
        sta DX
        sta DY
        jsr DRAW
        lda #FRAMETIME
        sta TIMER1024

```

```

FRAME:  bit TIMEOUT    ; wait for the next 1/50 s
        bpl FRAME
        lda #FRAMETIME
        sta TIMER1024
        jsr DRAW      ; erase the ball (EOR it again)

```

```

        clc          ; x = x + dx

```

```

        lda BX
        adc DX
        sta BX
        lda BX+1
        adc DX+1
        sta BX+1

```

```

        clc          ; y = y + dy

```

```

        lda BY
        adc DY
        sta BY

```

```

        lda BX        ; at the left edge? go right

```

```

        ora BX+1
        bne @notleft
        lda #2
        sta DX
        lda #0
        sta DX+1

```

```

@notleft:
        lda BX        ; at the right edge? go left

```

```

        cmp #<MAXX
        bne @notright
        lda BX+1
        cmp #>MAXX
        bne @notright
        lda #<-2
        sta DX
        lda #>-2
        sta DX+1

```

```

@notright:
        lda BY        ; at the top? go down
        bne @nottop

```

```

    lda #2
    sta DY
@nottop:
    lda BY          ; at the bottom? go up
    cmp #MAXY
    bne @notbottom
    lda #<-2
    sta DY
@notbottom:
    jsr DRAW        ; draw it in its new place
    jmp FRAME

; EOR the ball onto the screen at (BX, BY).
DRAW:  lda #0        ; ADDR = SCREEN + BY x 40 + BX / 8, as in problem 39
        sta ADDR+1
        lda BY
        asl a
        rol ADDR+1
        asl a
        rol ADDR+1
        clc
        adc BY
        bcc @x5
        inc ADDR+1
@x5:   asl a
        rol ADDR+1
        asl a
        rol ADDR+1
        asl a
        rol ADDR+1
        sta ADDR
        lda BX+1
        lsr a
        lda BX
        ror a
        lsr a
        lsr a
        clc
        adc ADDR
        sta ADDR
        lda ADDR+1
        adc #>SCREEN
        sta ADDR+1

        lda BX
        and #7
        sta SHIFT
        ldx #0        ; X = row of the shape
@row:  lda SHAPE,x
        sta LEFT
        lda #0
        sta RIGHT
        ldy SHIFT
        beq @shifted
@shift: lsr LEFT      ; shift the 16 bits LEFT:RIGHT right by one
        ror RIGHT
        dey
        bne @shift
@shifted:
        ldy #0        ; Y = 0 here either way

```

```

lda (ADDR),y
eor LEFT
sta (ADDR),y
iny
lda (ADDR),y
eor RIGHT
sta (ADDR),y
clc           ; next dot line
lda ADDR
adc #40
sta ADDR
bcc @same
inc ADDR+1
@same: inx
cpx #8
bne @row
rts

SHAPE: .byte %00111100
       .byte %01111110
       .byte %11111111
       .byte %11111111
       .byte %11111111
       .byte %11111111
       .byte %01111110
       .byte %00111100

```

44 Sketch pad

Problem 36's timing idea again. The frame timer starts before `GETKEY` and the movement, so the pen moves at a steady rate whatever the loop does.

```

.org $0200
SCREEN = $2000
TIMER1024 = $1707
TIMEOUT = $1707
STEPTIME = 19           ; (19 + 1) x 1024 us = 20.5 ms
XC = $30                ; pen position, as for PLOT
YC = $32
ADDR = $33
PTR = $20               ; for CLEAR

start: lda #0
      sta PADD           ; keypad columns are inputs, for GETKEY
      lda #<160
      sta XC
      lda #>160
      sta XC+1
      lda #100
      sta YC
      jsr CLEAR

@step: jsr PLOT          ; the pen leaves a dot wherever it is
      lda #STEPTIME
      sta TIMER1024
      jsr GETKEY
      cmp #0
      bne @up
      jsr CLEAR

```

```

    jmp @wait
@up:  cmp #9
      bne @down
      lda YC
      beq @wait      ; already at the top
      dec YC
      jmp @wait
@down: cmp #1
       bne @left
       lda YC
       cmp #199
       beq @wait      ; already at the bottom
       inc YC
       jmp @wait
@left: cmp #4
       bne @right
       lda XC
       ora XC+1
       beq @wait      ; x = 0
       lda XC          ; 16-bit decrement
       bne @nb
       dec XC+1
@nb:  dec XC
      jmp @wait
@right: cmp #6
        bne @wait
        lda XC
        cmp #<319
        bne @inc
        lda XC+1
        cmp #>319
        beq @wait      ; x = 319
@inc:  inc XC          ; 16-bit increment
       bne @wait
       inc XC+1
@wait: bit TIMEOUT
       bpl @wait
       jmp @step

; CLEAR (problem 37 with A = 0): not repeated here

; PLOT (problem 39): not repeated here

```

Appendix · 6502 instruction summary

Flags: N negative, V overflow, Z zero, C carry. “NZ” means the instruction sets N and Z from its result.

Instr.	Operation	Flags
ADC	$A = A + M + C$	NVZC
AND	$A = A \text{ and } M$	NZ
ASL	$C \leftarrow [7...0] \leftarrow 0$	NZC
BCC	branch if $C = 0$	
BCS	branch if $C = 1$	
BEQ	branch if $Z = 1$	
BIT	$Z = (A \text{ and } M) = 0$; $N = M7$, $V = M6$	NVZ
BMI	branch if $N = 1$	
BNE	branch if $Z = 0$	
BPL	branch if $N = 0$	
BRK	interrupt through IRQ vector	B, I
BVC	branch if $V = 0$	
BVS	branch if $V = 1$	
CLC	$C = 0$	C
CLD	binary mode	D
CLI	enable IRQ	I
CLV	$V = 0$	V
CMP	$A - M$, flags only	NZC
CPX	$X - M$, flags only	NZC
CPY	$Y - M$, flags only	NZC
DEC	$M = M - 1$	NZ
DEX	$X = X - 1$	NZ
DEY	$Y = Y - 1$	NZ
EOR	$A = A \text{ xor } M$	NZ
INC	$M = M + 1$	NZ
INX	$X = X + 1$	NZ
INY	$Y = Y + 1$	NZ
JMP	go to address	

Instr.	Operation	Flags
JSR	push return address - 1, go	
LDA	$A = M$	NZ
LDX	$X = M$	NZ
LDY	$Y = M$	NZ
LSR	$0 \rightarrow [7...0] \rightarrow C$	NZC
NOP	nothing	
ORA	$A = A \text{ or } M$	NZ
PHA	push A	
PHP	push P	
PLA	pull A	NZ
PLP	pull P	all
ROL	$C \leftarrow [7...0] \leftarrow C$	NZC
ROR	$C \rightarrow [7...0] \rightarrow C$	NZC
RTI	pull P, pull PC	all
RTS	pull PC, add 1	
SBC	$A = A - M - (1 - C)$	NVZC
SEC	$C = 1$	C
SED	decimal mode	D
SEI	disable IRQ	I
STA	$M = A$	
STX	$M = X$	
STY	$M = Y$	
TAX	$X = A$	NZ
TAY	$Y = A$	NZ
TSX	$X = S$	NZ
TXA	$A = X$	NZ
TXS	$S = X$	
TYA	$A = Y$	NZ